



Pixel Predators

Plant Disease Classification

ITAI 1378 | Computer Vision and AI | Tier 1

One-line pitch

Upload a leaf photo. Get a crop-condition prediction, a HEALTHY/DISEASED status, and a confidence score.

TEAM MEMBERS

Alexia Y Chavez · Elham Timory · Tobias Chow



THE PROBLEM

Plant disease is costly, slow to detect, and easy to miss

Every year, plant pests and diseases cause major crop losses. Early detection can help growers respond before damage spreads.

GLOBAL LOSSES

>\$220B

annual losses from plant pests and diseases

PRODUCTION RISK

Up to 40%

of crop production lost to pests and diseases

CURRENT PROCESS

Manual

scouting is slow and depends on expert access

Who cares

Farmers, agronomists, food-security groups, and crop insurance teams.

What it solves

Fast preliminary screening from a leaf photo, so growers can make earlier treatment decisions.

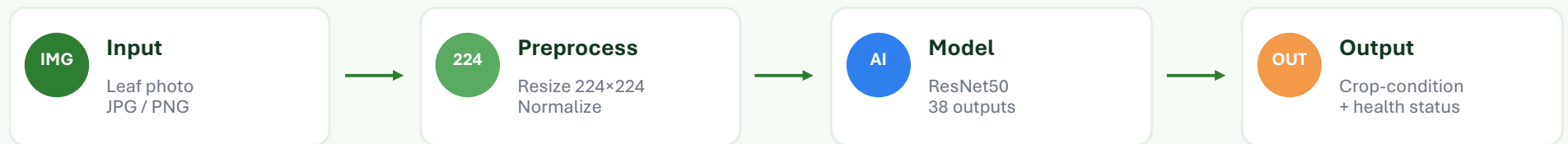
Sources: FAO Plant Production and Protection facts and figures; USDA ARS.



WHAT YOU BUILT

A 38-class classifier with a simple health-status summary

The final proof of concept takes a single leaf image and returns a crop-condition class, HEALTHY/DISEASED status, and confidence score.



Example output

Predicted class: Tomato__Early_blight
Health status: DISEASED
Confidence:96.7%

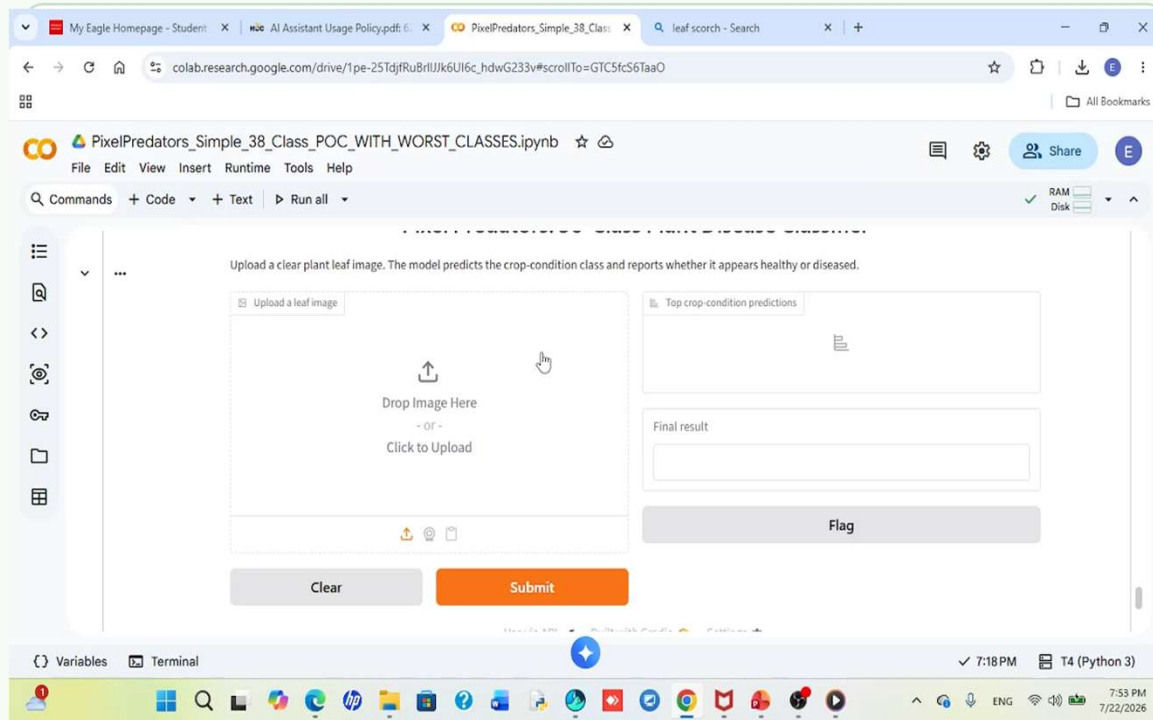
Application layer

Gradio upload page where a user selects a leaf image and sees the top predictions and report.



DEMO

Live walkthrough of the working proof of concept



Demo sequence

1. Upload

Use an unseen PlantVillage image and at least one phone photo.

2. Predict

Show the top crop-condition class returned by ResNet50.

3. Explain

Show HEALTHY/DISEASED status and confidence score.

4. Failure case

Show where the model struggles and explain why.



Technique, model architecture, and framework as built

CV technique

Image classification using transfer learning. One image goes in and one crop-condition class comes out.

Model

ResNet50 pretrained on ImageNet. The original final layer is replaced with a new 38-class classifier.

Training design

Stage 1 trains the classifier head. Stage 2 fine-tunes the final ResNet block with a smaller learning rate.

Framework

PyTorch + torchvision for training, Hugging Face/Kaggle for data access, and Gradio for the demo UI.

Why this fits Tier 1: pretrained image classification plus application logic. It is realistic for free Colab and the course timeline.



PlantVillage images, labels, cleaning, and preprocessing

PlantVillage Dataset

Public dataset via Kaggle / TensorFlow Datasets / PlantVillage mirrors

- ~54K healthy and diseased leaf images
- 38 crop-condition classes across 14 crop species
- Labels include crop name + disease or healthy status
- Mostly controlled leaf photos, so real-world testing matters
- Used balanced per-class subset if Colab limits required it

Preprocessing

- 01 **Resize — 224 × 224 px**
- 02 **Augment — training only: flip, rotation, color jitter**
- 03 **Normalize — ImageNet mean and standard deviation**
- 04 **Split — train / validation / held-out test**



Measured performance compared with the Blueprint targets

Metric	Blueprint target	Actual final result	Status
Top-1 test accuracy	90%+	97.71%	Met
Macro F1	≥ 0.88 avg.	97.73%	Met
Inference speed	< 1 sec / image	0.0095 seconds	Met
Worst classes	Reviewed	Tomato__Early_blight	
Error samples	Failure case included	Begonia	





Engineering choices made during the build

1

Prediction output

Kept the 38-class crop-condition prediction, then added a HEALTHY/DISEASED summary so the result is easier to understand.

2

Dataset access

Used a PlantVillage mirror that loaded reliably in Colab when one Hugging Face configuration caused problems.

3

Training scope

Used balanced per-class sampling if free Colab time or memory became a limit. This kept the proof of concept realistic.

4

Demo plan

Added a real-world failure leaf example to show how the model handles images outside PlantVillage.

The hardest problems and how we worked around them

Dataset loader issue

Challenge: Some PlantVillage sources/configurations did not load cleanly in Colab.

Fix: Used a reliable PlantVillage mirror and documented the exact source in the README.

Compute limits

Challenge: Full 38-class training can be slow on free Colab, especially with a large dataset.

Fix: Used transfer learning, frozen layers first, balanced sampling, checkpoints, and modest batch size.

Real-world domain gap

Challenge: Phone photos can have complex backgrounds, shadows, hands, pots, and unfamiliar plant species.

Fix: Tested outside PlantVillage and included a failure leaf case instead of hiding it.



Failure leaf example

A healthy polka dot begonia may be flagged as diseased because natural white spots look like symptoms to the model.



What we learned and what we would build next

Lessons learned

- The model target must match the actual application output.
- Accuracy alone is not enough; F1 and error examples reveal more.
- Test data must stay separate from training and tuning.
- Failure cases are useful because they show real limitations.
- Transfer learning made the project possible on free compute.

Next steps

- Collect more real-world phone images in outdoor and indoor settings.
- Add healthy ornamental plants with natural markings.
- Compare ResNet50 with EfficientNet or Vision Transformer models.
- Add clearer user guidance: “preliminary screen, not diagnosis.”
- Improve the app with history, image examples, and confidence warnings.



LINKS

Demo video and GitHub repository

Demo Video (3–5 min)

On slide 4

Shows real input → model output, narration, one successful example, and one failure case with explanation.

GitHub Repository

https://github.com/TobiasC05/ITAI1378_PlantDiseaseClassification_PDC

Contains README, notebook/src code, requirements.txt, AI usage log, presentation PDF, and results folder.



CONCLUSION

From a leaf photo to an early warning

Pixel Predators built a transfer-learning classifier that turns a single leaf photo into a fast preliminary plant-condition screen.

Working demo

Gradio app runs end to end

Public repo

README, code, results, and AI log

Demo video

Real input/output plus a failure case

Thank you — questions welcome.

Alexia Y Chavez · Elham Timory · Tobias Chow

